

Module 8: Filtering Data with WHERE Clause

The **WHERE** clause is one of the most important SQL concepts. It allows you to retrieve only the records that meet specific conditions instead of returning every row from a table.



Introduction

Imagine a table with 1 million employee records.

If you want to find:

- Employees earning more than ₹50,000
- Employees from Delhi
- Employees hired after 2024

You do not want all records.

You need filtering.

The **WHERE** clause enables SQL to filter rows based on conditions.



Learning Objectives

After completing this module, you will be able to:

- ✓ Filter records using WHERE
 - ✓ Use comparison operators
 - ✓ Use logical operators
 - ✓ Work with BETWEEN
 - ✓ Use IN operator
 - ✓ Use LIKE and wildcards
 - ✓ Handle NULL values correctly
 - ✓ Write complex filtering queries
-



Table of Contents

1. What is WHERE?
2. WHERE Syntax

3. Comparison Operators
4. Logical Operators
5. BETWEEN Operator
6. IN Operator
7. LIKE Operator
8. Wildcards
9. NULL Handling
10. Combining Conditions
11. Query Execution Order
12. Common Mistakes
13. Best Practices
14. Summary
15. Practice Questions

1 What is WHERE?

The **WHERE** clause filters records based on a condition.

Without WHERE:

```
SELECT *  
FROM Employees;
```

Returns:

```
All rows
```

With WHERE:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Returns:

```
Only employees earning more than 50000
```

Why WHERE Matters

Used in:

- Reports
- Dashboards
- Analytics
- Data Cleaning
- Business Intelligence

Nearly every real-world SQL query uses WHERE.

2 WHERE Syntax

Basic syntax:

```
SELECT ColumnName
FROM TableName
WHERE Condition;
```

Example

```
SELECT *
FROM Employees
WHERE EmployeeID = 101;
```

Output:

EmployeeID	EmployeeName
101	Kunal

Example Table

EmployeeID	EmployeeName	Salary	City
101	Kunal	45000	Delhi
102	Rahul	55000	Mumbai
103	Aman	60000	Delhi

3 Comparison Operators

Comparison operators compare values.

Operator	Meaning
=	Equal To
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To
<>	Not Equal To
!=	Not Equal To

Equal To (=)

```
SELECT *  
FROM Employees  
WHERE City = 'Delhi';
```

Output:

EmployeeID	EmployeeName
101	Kunal
103	Aman

Greater Than (>)

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Output:

EmployeeName	Salary
Rahul	55000
Aman	60000

Less Than (<)

```
SELECT *  
FROM Employees  
WHERE Salary < 50000;
```

Output:

<u>EmployeeName</u>
Kunal

Greater Than or Equal To (>=)

```
SELECT *  
FROM Employees  
WHERE Salary >= 55000;
```

Less Than or Equal To (<=)

```
SELECT *  
FROM Employees  
WHERE Salary <= 50000;
```

Not Equal To

```
SELECT *  
FROM Employees  
WHERE City <> 'Delhi';
```

Or

```
SELECT *  
FROM Employees
```

```
WHERE City != 'Delhi';
```



Logical Operators

Used to combine multiple conditions.

AND

Both conditions must be TRUE.

Example:

```
SELECT *  
FROM Employees  
WHERE City = 'Delhi'  
AND Salary > 50000;
```

Result:

EmployeeName
Aman

OR

At least one condition must be TRUE.

Example:

```
SELECT *  
FROM Employees  
WHERE City = 'Delhi'  
OR Salary > 55000;
```

NOT

Reverses condition.

Example:

```
SELECT *  
FROM Employees  
WHERE NOT City = 'Delhi';
```

Logical Operator Truth Table

Condition A	Condition B	AND	OR
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

5 BETWEEN Operator

Used to find values within a range.

Syntax

```
WHERE ColumnName  
BETWEEN Value1 AND Value2
```

Example

```
SELECT *  
FROM Employees  
WHERE Salary  
BETWEEN 40000 AND 60000;
```

Equivalent to:

```
WHERE Salary >= 40000  
AND Salary <= 60000
```

Date Example

```
SELECT *  
FROM Orders  
WHERE OrderDate  
BETWEEN '2025-01-01'  
AND '2025-12-31';
```

Use Cases

- Salary ranges
- Age ranges
- Date ranges
- Product prices

IN Operator

Used to match multiple values.

Without IN:

```
WHERE City = 'Delhi'  
OR City = 'Mumbai'  
OR City = 'Pune'
```

With IN:


```
WHERE City IN  
(  
    'Delhi',  
    'Mumbai',  
    'Pune'  
)
```

Example

```
SELECT *  
FROM Employees  
WHERE City IN  
(  
    'Delhi',  
    'Mumbai'  
);
```

NOT IN

```
SELECT *  
FROM Employees  
WHERE City NOT IN  
(  
    'Delhi',  
    'Mumbai'  
);
```

Benefits

- ✓ Cleaner code
- ✓ Easier maintenance

LIKE Operator

Used for pattern matching.

Syntax

```
WHERE ColumnName  
LIKE Pattern
```

Example:

```
SELECT *  
FROM Employees  
WHERE EmployeeName LIKE 'K%';
```

Returns:

```
Kunal  
Krishna  
Kartik
```

Use Cases

- Customer search
- Product search
- Name filtering



Wildcards

Wildcards work with LIKE.

% Wildcard

Represents:

```
Zero or more characters
```

Starts With

```
WHERE EmployeeName LIKE 'K%'
```

Examples:

```
Kunal  
Kartik  
Krishna
```

Ends With

```
WHERE EmployeeName LIKE '%a'
```

Examples:

```
Priya  
Neha
```

Contains

```
WHERE EmployeeName LIKE '%an%'
```

Examples:

```
Ankit  
Kunal Anand
```

_ Wildcard

Represents:

Exactly one character

Example:

```
WHERE Name LIKE '_a%'
```

Matches:

Rahul
Karan

Multiple Wildcard Examples

Pattern	Meaning
K%	Starts with K
%a	Ends with a
%an%	Contains an
_a%	Second letter a



NULL Handling

NULL means:

Unknown
Missing
Not Available

Wrong Way

```
WHERE Salary = NULL
```

Never works.

Correct Way

```
WHERE Salary IS NULL
```

Example:

```
SELECT *  
FROM Employees  
WHERE Salary IS NULL;
```

Non-NULL Values

```
SELECT *  
FROM Employees  
WHERE Salary IS NOT NULL;
```

Why?

SQL treats NULL differently from normal values.

10 Combining Conditions

Complex filtering often combines operators.

Example:

```
SELECT *  
FROM Employees  
WHERE City = 'Delhi'  
AND Salary > 50000;
```

Example:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary BETWEEN 40000 AND 80000;
```

Example:

```
SELECT *  
FROM Employees  
WHERE City IN ('Delhi', 'Mumbai')  
AND EmployeeName LIKE 'K%';
```

Parentheses

Improve clarity.

Example:

```
SELECT *  
FROM Employees  
WHERE  
(  
    City = 'Delhi'  
    OR City = 'Mumbai'  
)  
AND Salary > 50000;
```

1**1**

Query Execution Order

Consider:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Execution:

1. FROM
2. WHERE
3. SELECT

Understanding execution order helps with:

- Query optimization
 - Debugging
 - Performance tuning
-

1 2 Common Mistakes

Using = with NULL

Bad:

```
WHERE Salary = NULL
```

Correct:

```
WHERE Salary IS NULL
```

Missing Quotes

Bad:

```
WHERE City = Delhi
```

Correct:

```
WHERE City = 'Delhi'
```

Wrong Operator

Bad:

```
WHERE Salary => 50000
```

Correct:

```
WHERE Salary >= 50000
```

Ignoring Parentheses

Bad:

```
WHERE City='Delhi'  
OR City='Mumbai'  
AND Salary>50000
```

Better:

```
WHERE  
(  
  City='Delhi'  
  OR City='Mumbai'  
)  
AND Salary>50000
```

1 **3** Best Practices

Use Specific Filters

Good:

```
WHERE EmployeeID = 101
```

Use IN for Multiple Values

Good:

```
WHERE City IN ('Delhi', 'Mumbai')
```

Use BETWEEN for Ranges

Good:

```
WHERE Salary BETWEEN 30000 AND 50000
```

Handle NULL Correctly

Use:

```
IS NULL
```

and

```
IS NOT NULL
```

Use Parentheses

Improves readability.

Example of Professional Query

```
SELECT EmployeeID,  
       EmployeeName,  
       Salary  
FROM Employees  
WHERE  
(  
    City IN ('Delhi', 'Mumbai')  
)
```

```
AND Salary BETWEEN 40000 AND 80000  
AND EmployeeName LIKE 'K%';
```



Summary

In this module, you learned:

- ✓ WHERE Clause
- ✓ Comparison Operators
- ✓ Logical Operators
- ✓ BETWEEN
- ✓ IN
- ✓ LIKE
- ✓ Wildcards
- ✓ NULL Handling
- ✓ Complex Filtering
- ✓ Query Execution Order



Practice Questions

Theory

1. What is the WHERE clause?
2. Difference between AND and OR?
3. What is BETWEEN?
4. What is IN?
5. What is LIKE?
6. What does % represent?
7. What does _ represent?
8. Why does = NULL not work?
9. Difference between IS NULL and IS NOT NULL?
10. Why use parentheses?

Practical Exercises

Task 1

Display employees:

```
Salary > 50000
```

Task 2

Display employees from:

```
Delhi  
Mumbai
```

using IN.

Task 3

Display employees whose names start with:

```
K
```

Task 4

Display employees whose salary is between:

```
40000 and 70000
```

Task 5

Find records where salary is NULL.

Task 6

Find employees:

```
City = Delhi  
Salary > 50000
```

using AND.

Challenge Project

Using an Employees table:

Generate reports for:

- Employees earning above average salary
- Employees from specific cities
- Employees with names matching patterns
- Employees hired within date ranges

Use:

- WHERE
- BETWEEN
- IN
- LIKE
- NULL Handling



Next Module

→ Module 9: ORDER BY Clause

Topics Covered:

- Sorting Data
- ASC and DESC
- Sorting by Multiple Columns
- ORDER BY with Aliases
- ORDER BY with Calculated Columns
- Sorting NULL Values
- Performance Considerations